

PYTHON • 5 SESSION ASSESSMENT

Python Concepts & Problem-Solving Workbook

75 source-based questions across MCQs, 2-mark tests, programming problems, real applications and programming-level thinking.

5 SESSIONS	75 QUESTIONS	75 FULL ANSWERS
Functional Python HTTP & JSON Web Scraping Modules & Regex Pandas & CSV	25 MCQs 10 Two-Mark 20 Problems 10 Real Apps 10 Thinking	Solutions Explanations Reasoning Edge cases Interview notes

HOW TO USE THIS WORKBOOK

Work session by session. Attempt each question before opening the answer section. Use the pseudocode panel as a planning guide, not as a solution. Requirements and examples define the expected behavior. The answer section is intentionally placed at the end so the practice portion remains independent.

Source basis: s10.py, s11.py, s12.py, s13.py, s14.py, and the supplied marks.csv dataset reference.

WORKBOOK MAP

Five sessions, one consistent assessment format

S1 FUNCTIONAL PYTHON TOOLS	s11.py	List comprehension, Conditional / nested ternary expression, lambda functions, filter(), map(), map/filter pipeline
S2 HTTP, JSON & FILE HANDLING	s12.py	requests.get(), response.text, json.loads(), API data extraction, file modes, open() / write() / close()
S3 JSON & WEB SCRAPING	s13.py	json.dumps() / json.loads(), BeautifulSoup, html.parser, find_all(), tag.text, replace() / numeric cleaning, writing scraped data
S4 MODULES, TIME & REGEX VALIDATION	s10.py	import, from ... import, aliases, time / sleep(), time.localtime(), re.search(), password validation, retry / lockout logic
S5 PANDAS & CSV DATA ANALYSIS	s14.py, marks.csv	pathlib.Path, pandas.read_csv(), head() / tail(), columns, fillna() / dropna(), derived columns, apply(lambda), loc / iloc, duplicated() / drop_duplicates(), to_csv()

QUESTION DISTRIBUTION

MCQs	25	5
Two-mark questions	10	2
Easy-to-hard programming problems	20	4
Real-application problems	10	2
Programming-level thinking	10	2
TOTAL	75	15

CLICKABLE INDEX

Go directly to any session, question or concept

NAVIGATION

Click a question title to jump directly to it. Every question and answer page includes a Back to Index link. Click a concept to jump to its session notes.

S1 - FUNCTIONAL PYTHON TOOLS

List comprehensions, ternary expressions, lambda, map(), filter(), and pipelines

- [List comprehension](#)
- [Conditional / nested ternary expression](#)
- [lambda functions](#)
- [filter\(\)](#)
- [map\(\)](#)
- [map/filter pipeline](#)

Question 1: List comprehension output	MCQ	Easy
Question 2: Ternary selection	MCQ	Easy
Question 3: lambda expression	MCQ	Easy
Question 4: filter() result	MCQ	Easy
Question 5: map vs filter	MCQ	Medium
Question 6: Write a conditional list comprehension	2MARK	Easy
Question 7: Explain the final list	2MARK	Medium
Question 8: Transform qualifying scores	PROBLEM	Easy
Question 9: Create a conditional transformed list	PROBLEM	Easy
Question 10: Build a reusable threshold pipeline	PROBLEM	Medium
Question 11: Choose map or filter correctly	PROBLEM	Medium
Question 12: Customer price eligibility report	REAL	Medium
Question 13: Attendance status generator	REAL	Medium
Question 14: Which tool should you choose?	THINK	Medium
Question 15: Debug the pipeline	THINK	Medium

S2 - HTTP, JSON & FILE HANDLING

requests, response text, JSON conversion, API extraction, file modes and writing

- [requests.get\(\)](#)
- [response.text](#)
- [json.loads\(\)](#)
- [API data extraction](#)
- [file modes](#)
- [open\(\) / write\(\) / close\(\)](#)

Question 16: requests.get() content	MCQ	Easy
Question 17: json.loads()	MCQ	Easy
Question 18: File mode	MCQ	Easy

Question 19: API titles with map	MCQ	Medium
Question 20: Reading after write	MCQ	Easy
Question 21: Parse a JSON string	2MARK	Easy
Question 22: List the file modes	2MARK	Easy
Question 23: Extract API product titles	PROBLEM	Easy
Question 24: Write API data to a text file	PROBLEM	Easy
Question 25: Read a simple API temperature	PROBLEM	Easy
Question 26: Create a small API report	PROBLEM	Medium
Question 27: Product API reporting job	REAL	Medium
Question 28: City weather summary	REAL	Medium
Question 29: What should happen when an API returns bad JSON?	THINK	Medium
Question 30: Choose the right file mode	THINK	Easy

S3 - JSON & WEB SCRAPING

JSON serialization, BeautifulSoup parsing, selectors, text extraction, cleaning and persistence

- `json.dumps()` / `json.loads()`
- BeautifulSoup
- `html.parser`
- `find_all()`
- `tag.text`
- `replace()` / numeric cleaning
- writing scraped data

Question 31: BeautifulSoup parser	MCQ	Easy
Question 32: <code>find_all()</code>	MCQ	Easy
Question 33: <code>tag.text</code>	MCQ	Easy
Question 34: Currency cleaning	MCQ	Medium
Question 35: Persist scraped titles	MCQ	Easy
Question 36: Extract all titles	2MARK	Easy
Question 37: Clean a price string	2MARK	Easy
Question 38: Extract and print product titles	PROBLEM	Easy
Question 39: Filter prices from scraped text	PROBLEM	Medium
Question 40: Save scraped data with numbering	PROBLEM	Easy
Question 41: Count matching products	PROBLEM	Medium
Question 42: Price-monitor report	REAL	Medium
Question 43: Product title export	REAL	Easy
Question 44: Make the scraper resilient	THINK	Medium
Question 45: Separate raw and cleaned values	THINK	Medium

S4 - MODULES, TIME & REGEX VALIDATION

imports, aliases, `sleep()`, local time, regular expressions and validation logic

- `import`
- `from ... import`
- aliases

- `time / sleep()`
- `time.localtime()`
- `re.search()`
- password validation
- `retry / lockout` logic

Question 46: Import alias	MCQ	Easy
Question 47: <code>sleep()</code>	MCQ	Easy
Question 48: Regex character class	MCQ	Easy
Question 49: Retry counter	MCQ	Medium
Question 50: <code>localtime()</code>	MCQ	Easy
Question 51: Alias an imported function	2MARK	Easy
Question 52: One regex requirement	2MARK	Easy
Question 53: Password rule validator	PROBLEM	Medium
Question 54: Retry with sleep mode	PROBLEM	Medium
Question 55: Format local time	PROBLEM	Easy
Question 56: Use a user-defined module cleanly	PROBLEM	Easy
Question 57: Account lockout guard	REAL	Hard
Question 58: Reusable time utility module	REAL	Medium
Question 59: Design the regex checks	THINK	Medium
Question 60: Module import choice	THINK	Medium

S5 - PANDAS & CSV DATA ANALYSIS

Path, `read_csv()`, inspection, missing data, derived columns, filtering, duplicates and export

- `pathlib.Path`
- `pandas.read_csv()`
- `head()` / `tail()`
- `columns`
- `fillna()` / `dropna()`
- derived columns
- `apply(lambda)`
- `loc` / `iloc`
- `duplicated()` / `drop_duplicates()`
- `to_csv()`

Question 61: <code>Path(__file__).parent</code>	MCQ	Medium
Question 62: <code>head()</code> / <code>tail()</code>	MCQ	Easy
Question 63: <code>fillna</code> vs <code>dropna</code>	MCQ	Easy
Question 64: <code>loc</code> vs <code>iloc</code>	MCQ	Medium
Question 65: <code>apply(lambda)</code>	MCQ	Medium
Question 66: Create Total column	2MARK	Easy
Question 67: Difference between <code>loc</code> and <code>iloc</code>	2MARK	Medium
Question 68: Clean and classify marks	PROBLEM	Medium
Question 69: Filter failing Maths rows	PROBLEM	Medium
Question 70: Remove duplicate records	PROBLEM	Easy
Question 71: Export cleaned results	PROBLEM	Easy

Question 72: Student performance report	REAL	Hard
Question 73: Academic failure filter	REAL	Medium
Question 74: Fill or drop missing data?	THINK	Hard
Question 75: Vectorized column calculation vs row loop	THINK	Hard

ANSWER INDEX

Jump directly to the complete answers

S1 - Answer block

- Answer 1: List comprehension output
- Answer 2: Ternary selection
- Answer 3: lambda expression
- Answer 4: filter() result
- Answer 5: map vs filter
- Answer 6: Write a conditional list comprehension
- Answer 7: Explain the final list
- Answer 8: Transform qualifying scores
- Answer 9: Create a conditional transformed list
- Answer 10: Build a reusable threshold pipeline
- Answer 11: Choose map or filter correctly
- Answer 12: Customer price eligibility report
- Answer 13: Attendance status generator
- Answer 14: Which tool should you choose?
- Answer 15: Debug the pipeline

S2 - Answer block

- Answer 16: requests.get() content
- Answer 17: json.loads()
- Answer 18: File mode
- Answer 19: API titles with map
- Answer 20: Reading after write
- Answer 21: Parse a JSON string
- Answer 22: List the file modes
- Answer 23: Extract API product titles
- Answer 24: Write API data to a text file
- Answer 25: Read a simple API temperature
- Answer 26: Create a small API report
- Answer 27: Product API reporting job
- Answer 28: City weather summary
- Answer 29: What should happen when an API returns bad JSON?
- Answer 30: Choose the right file mode

S3 - Answer block

- Answer 31: BeautifulSoup parser
- Answer 32: find_all()
- Answer 33: tag.text
- Answer 34: Currency cleaning
- Answer 35: Persist scraped titles
- Answer 36: Extract all titles
- Answer 37: Clean a price string
- Answer 38: Extract and print product titles
- Answer 39: Filter prices from scraped text
- Answer 40: Save scraped data with numbering
- Answer 41: Count matching products
- Answer 42: Price-monitor report
- Answer 43: Product title export
- Answer 44: Make the scraper resilient
- Answer 45: Separate raw and cleaned values

S4 - Answer block

- Answer 46: Import alias
- Answer 47: sleep()
- Answer 48: Regex character class
- Answer 49: Retry counter
- Answer 50: localtime()

- Answer 51: Alias an imported function
- Answer 52: One regex requirement
- Answer 53: Password rule validator
- Answer 54: Retry with sleep mode
- Answer 55: Format local time
- Answer 56: Use a user-defined module cleanly
- Answer 57: Account lockout guard
- Answer 58: Reusable time utility module
- Answer 59: Design the regex checks
- Answer 60: Module import choice

S5 - Answer block

- Answer 61: `Path(__file__).parent`
- Answer 62: `head()` / `tail()`
- Answer 63: `fillna` vs `dropna`
- Answer 64: `loc` vs `iloc`
- Answer 65: `apply(lambda)`
- Answer 66: Create Total column
- Answer 67: Difference between `loc` and `iloc`
- Answer 68: Clean and classify marks
- Answer 69: Filter failing Maths rows
- Answer 70: Remove duplicate records
- Answer 71: Export cleaned results
- Answer 72: Student performance report
- Answer 73: Academic failure filter
- Answer 74: Fill or drop missing data?
- Answer 75: Vectorized column calculation vs row loop

SESSION 1

FUNCTIONAL PYTHON TOOLS

List comprehensions, ternary expressions, lambda, map(), filter(), and pipelines

Source files: s11.py

SESSION FORMAT

15 questions in this session: 5 MCQs, 2 two-mark questions, 4 programming problems, 2 real-application problems and 2 programming-level thinking problems. Concept Notes follow the practice set.

QUESTION 1 · MCQ

List comprehension output

Concept: List comprehension Source: s11.py

QUESTION

What is printed by the code?

- A. [1, 2, 3]
- B. [2, 4, 6]
- C. [3, 6, 9]
- D. Error

SKILL TESTED: List comprehension

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

START with a list
FOR each item x
APPEND x*2

QUESTION 2 · MCQ

Ternary selection

Concept: Conditional expression Source: s11.py

QUESTION

Which value is assigned to res?

- A. Vote
- B. Wait
- C. True
- D. Error

SKILL TESTED: Conditional expression

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

IF condition true -> first value
ELSE -> second value

QUESTION 3 · MCQ

lambda expression

Concept: lambda Source: s11.py

QUESTION

What does f("Dev") return?

- A. Dev
- B. Hello
- C. Hello Dev
- D. None

SKILL TESTED: lambda

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

TAKE name
BUILD greeting string
RETURN it

QUESTION 4 · MCQ**filter() result**

Concept: filter() Source: s11.py

QUESTION

Which list is produced?

- A. [3, 8]
- B. [20, 21]
- C. [400, 441]
- D. [True, True]

SKILL TESTED: filter()

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
FOR each n
IF n >= 20 -> KEEP
ELSE discard
```

QUESTION 5 · MCQ**map vs filter**

Concept: map() / filter() Source: s11.py

QUESTION

Which statement is correct?

- A. map only filters
- B. filter squares values
- C. filter selects and map transforms
- D. They are identical

SKILL TESTED: map() / filter()

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
FILTER decides which items survive
MAP changes each surviving item
```

QUESTION 6 · 2MARK**Write a conditional list comprehension**

Concept: List comprehension Source: s11.py

PROBLEM STATEMENT

Write a list comprehension that keeps even numbers unchanged and squares odd numbers for [1,2,3,4].

REQUIREMENTS

Use one comprehension and a conditional expression.

EXAMPLE INPUT

```
[1, 2, 3, 4]
```

EXPECTED OUTPUT

```
[1, 2, 9, 4]
```

THINKING HINT

Put the conditional expression inside the comprehension.

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
FOR each value
IF even -> value
ELSE -> value squared
```

QUESTION 7 · 2MARK**Explain the final list**

Concept: map() + filter() Source: s11.py

PROBLEM STATEMENT

What is the result of `list(map(lambda m: m**2, filter(lambda a: a >= 20, [3,35,62,2,72,18,9])))`?

REQUIREMENTS

Show the filtered values and then their squares.

EXAMPLE INPUT

```
[3, 35, 62, 2, 72, 18, 9]
```

EXPECTED OUTPUT

```
[1225, 3844, 5184]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

FILTER values $\geq 20 \rightarrow [35, 62, 72]$

MAP square $\rightarrow [1225, 3844, 5184]$

QUESTION 8 · PROBLEM**Transform qualifying scores**

Concept: map() + filter() Source: s11.py

PROBLEM STATEMENT

Given a list of scores, keep scores at least 50 and convert each passing score into a percentage label such as "Pass: 80%" using `map()`.

REQUIREMENTS

Use `filter()` for selection and `map()` for transformation. Return a list. Do not use a loop.

EXAMPLE INPUT

```
scores = [42, 55, 80, 37, 91]
```

EXPECTED OUTPUT

```
["Pass: 55%", "Pass: 80%", "Pass: 91%"]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

FILTER pass scores ≥ 50

MAP each retained score $\rightarrow$ Pass label

QUESTION 9 · PROBLEM**Create a conditional transformed list**

Concept: List comprehension + ternary Source: s11.py

PROBLEM STATEMENT

For values `[3,5,6,2,7,8,9]`, produce a list where even values stay unchanged and odd values are squared.

REQUIREMENTS

Use exactly one list comprehension and a conditional expression.

EXAMPLE INPUT

```
[3,5,6,2,7,8,9]
```

EXPECTED OUTPUT

```
[9, 25, 6, 2, 49, 8, 81]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

FOR each item

IF even $\rightarrow$ item

ELSE $\rightarrow$ item squared

QUESTION 10 · PROBLEM**Build a reusable threshold pipeline**

Concept: lambda + filter + map Source: s11.py

PROBLEM STATEMENT

Write a function that accepts a list and threshold, filters values at or above the threshold, then returns their squares.

REQUIREMENTS

Use filter() and map() with lambda expressions inside the function.

EXAMPLE INPUT

```
values=[10,25,30,15], threshold=20
```

EXPECTED OUTPUT

```
[625, 900]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
FILTER by threshold
MAP each retained value to square
RETURN list
```

QUESTION 11 · PROBLEM**Choose map or filter correctly**

Concept: map/filter design Source: s11.py

PROBLEM STATEMENT

You receive product quantities [0,2,5,1]. Return only products whose quantity is positive, then convert each retained quantity to a stock label such as "5 units".

REQUIREMENTS

Use filter() first and map() second.

EXAMPLE INPUT

```
[0,2,5,1]
```

EXPECTED OUTPUT

```
["2 units", "5 units", "1 units"]
```

THINKING HINT

Selection comes before formatting.

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
FILTER quantity > 0
MAP quantity -> formatted label
```

QUESTION 12 · REAL**Customer price eligibility report**

Concept: filter + map Source: s11.py

PROBLEM STATEMENT

An online store stores prices and wants a short report of products that qualify for a discount.

REQUIREMENTS

Keep prices ≥ 1000 , then map them to labels "Eligible: ₹X". Preserve order and use a filter/map pipeline.

EXAMPLE INPUT

```
prices = [499, 1200, 999, 2500]
```

EXPECTED OUTPUT

```
Eligible: ₹1200
Eligible: ₹2500
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
FILTER price  $\geq 1000$ 
MAP price -> formatted report line
PRINT each line
```

QUESTION 13 · REAL**Attendance status generator**

Concept: lambda + map Source: s11.py

PROBLEM STATEMENT

A class system has attendance percentages. Convert each number to "Present" when it is 75 or more, otherwise "Short".

REQUIREMENTS

Use map() with a lambda. Do not use an explicit loop.

EXAMPLE INPUT

```
[82, 68, 91, 74]
```

EXPECTED OUTPUT

```
["Present", "Short", "Present", "Short"]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
MAP each attendance value
IF >= 75 -> Present
ELSE -> Short
```

QUESTION 14 · THINK**Which tool should you choose?**

Concept: map vs filter Source: s11.py

PROBLEM STATEMENT

A list contains [5, 12, 18, 22]. The requirement is: keep values ≥ 18 and replace each kept value by its square. Explain the correct tool order before coding.

REQUIREMENTS

State why filter must come before map.

EXAMPLE INPUT

```
[5,12,18,22]
```

EXPECTED OUTPUT

```
[324,484]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
SELECT -> FILTER >= 18
TRANSFORM -> MAP square
RETURN [324,484]
```

QUESTION 15 · THINK**Debug the pipeline**

Concept: filter + map Source: s11.py

PROBLEM STATEMENT

A student wrote: `list(filter(lambda x: x**2, [1,2,3]))`. Explain why this does not square the list and write the intended pipeline.

REQUIREMENTS

Explain the roles of predicate and transformer.

EXAMPLE INPUT

```
[1,2,3]
```

EXPECTED OUTPUT

```
Intended result after squaring: [1,4,9]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
FILTER expects a truth test
MAP is the transformation step
```

CONCEPT NOTES

FUNCTIONAL PYTHON TOOLS - What to remember

List comprehension

WHAT YOU SHOULD KNOW

A compact syntax for building a list from an iterable, optionally with a condition or conditional expression.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of List comprehension, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Ternary expression

WHAT YOU SHOULD KNOW

A one-line conditional expression that chooses one of two values based on a condition.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Ternary expression, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

lambda

WHAT YOU SHOULD KNOW

A small anonymous function expression, useful when a short function is needed inline.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of lambda, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

filter()

WHAT YOU SHOULD KNOW

Selects items for which the predicate evaluates to a truthy value.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of filter(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

map()

WHAT YOU SHOULD KNOW

Transforms each item by applying a function and returns a map iterator.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of map(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Pipeline

WHAT YOU SHOULD KNOW

Combining filter() for selection and map() for transformation separates responsibilities.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Pipeline, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Conditional / nested ternary expression

WHAT YOU SHOULD KNOW

Conditional / nested ternary expression is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Conditional / nested ternary expression, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

lambda functions

WHAT YOU SHOULD KNOW

lambda functions is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of lambda functions, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

map/filter pipeline

WHAT YOU SHOULD KNOW

map/filter pipeline is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of map/filter pipeline, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

SESSION 2

HTTP, JSON & FILE HANDLING

requests, response text, JSON conversion, API extraction, file modes and writing

Source files: s12.py

SESSION FORMAT

15 questions in this session: 5 MCQs, 2 two-mark questions, 4 programming problems, 2 real-application problems and 2 programming-level thinking problems. Concept Notes follow the practice set.

QUESTION 16 · MCQ

requests.get() content

Concept: requests Source: s12.py

QUESTION

What does response.text provide in the supplied API examples?

- A. A list
- B. A string containing response text
- C. A JSON dict automatically
- D. A file object

SKILL TESTED: requests

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

HTTP GET -> response object
USE response.text -> raw text

QUESTION 17 · MCQ

json.loads()

Concept: JSON conversion Source: s12.py

QUESTION

What does json.loads() do in the provided examples?

- A. Python object -> JSON text
- B. JSON text -> Python object
- C. File -> JSON file
- D. HTML -> DOM

SKILL TESTED: JSON conversion

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

JSON text -> Python object

QUESTION 18 · MCQ**File mode**

Concept: File handling Source: s12.py

QUESTION

Which file mode in the source is used to create/write demo.txt?

- A. r
- B. a
- C. w
- D. r+

SKILL TESTED: File handling

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**OPEN file in w mode
WRITE contents**QUESTION 19 · MCQ****API titles with map**

Concept: requests + map Source: s12.py

QUESTION

Which expression extracts every title from a JSON list of product dictionaries?

- A. list(filter(lambda x:x["title"],res))
- B. list(map(lambda x:x["title"],res))
- C. json.loads(res)
- D. res.text

SKILL TESTED: requests + map

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

MAP each dict -> item["title"]

QUESTION 20 · MCQ**Reading after write**

Concept: File handling Source: s12.py

QUESTION

Why must f.close() be called in the simple open/write example?

- A. To convert to JSON
- B. To sort the file
- C. To release the open file resource
- D. To make it read-only

SKILL TESTED: File handling

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

OPEN -> WRITE -> CLOSE

QUESTION 21 · 2MARK**Parse a JSON string**

Concept: json.loads() Source: s12.py

PROBLEM STATEMENT

Given text = '[{"name":"Dev"}]', write the expression that converts it into a Python object.

REQUIREMENTS

Use the JSON library operation shown in the source.

EXAMPLE INPUT

text = '[{"name":"Dev"}]'

EXPECTED OUTPUT

[{"name":"Dev"}] as a Python list of dicts

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**CALL json.loads(text)
STORE the Python object

QUESTION 22 · 2MARK**List the file modes**

Concept: File modes Source: s12.py

PROBLEM STATEMENT

Name the four modes explicitly shown in the source and state their purpose.

REQUIREMENTS

Keep the definitions concise.

EXAMPLE INPUT

```
t, r, w, a
```

EXPECTED OUTPUT

```
t=text/binary context as noted; r=read; w=write; a=append
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

REMEMBER r/read, w/write, a/append

QUESTION 23 · PROBLEM**Extract API product titles**

Concept: requests + JSON + map Source: s12.py

PROBLEM STATEMENT

Using a Python list representing API JSON data, return all product titles in order.

REQUIREMENTS

Assume requests.get(url).text has already been parsed with json.loads into a list of dictionaries.

EXAMPLE INPUT

```
[{"title": "Bag"}, {"title": "Shoes"}, {"title": "Watch"}]
```

EXPECTED OUTPUT

```
["Bag", "Shoes", "Watch"]
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
RECEIVE list of dicts
MAP item -> item[title]
RETURN list
```

QUESTION 24 · PROBLEM**Write API data to a text file**

Concept: requests + file writing Source: s12.py

PROBLEM STATEMENT

Suppose an API has already returned a list of product titles. Write them to demo.txt, one per line, preserving the order.

REQUIREMENTS

Open in w mode, write one line per title, and close the file.

EXAMPLE INPUT

```
["Bag", "Shoes", "Watch"]
```

EXPECTED OUTPUT

```
File contains three lines: Bag, Shoes, Watch
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
OPEN demo.txt in w
FOR each title -> write title + newline
CLOSE
```

QUESTION 25 • PROBLEM**Read a simple API temperature**

Concept: requests + JSON Source: s12.py

PROBLEM STATEMENT

Given a JSON string containing data[0]["temp"], parse it and print the temperature.

REQUIREMENTS

Use json.loads(); do not manually slice the JSON text.

EXAMPLE INPUT

```
{"data":[{"temp":28.4}]}
```

EXPECTED OUTPUT

```
28.4
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
PARSE JSON text
ACCESS data[0][temp]
PRINT value
```

QUESTION 26 • PROBLEM**Create a small API report**

Concept: requests + JSON + map Source: s12.py

PROBLEM STATEMENT

Given parsed API data with title and price fields, build report strings "Title - Price" using map().

REQUIREMENTS

Do not use a loop for the transformation itself.

EXAMPLE INPUT

```
[{"title":"Bag","price":50}, {"title":"Pen","price":10}]
```

EXPECTED OUTPUT

```
["Bag - 50", "Pen - 10"]
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
MAP each dict -> formatted string
```

QUESTION 27 • REAL**Product API reporting job**

Concept: requests + JSON + map + file Source: s12.py

PROBLEM STATEMENT

Build a small reporting script that takes API-like product JSON and creates a text report file.

REQUIREMENTS

Requirements: parse JSON text, extract title and price, write one report line per product, close the file.

EXAMPLE INPUT

```
[{"title":"Bag","price":50}, {"title":"Shoes","price":80}]
```

EXPECTED OUTPUT

```
Report file lines: Bag - 50 and Shoes - 80
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

```
PARSE -> TRANSFORM -> WRITE -> CLOSE
```

QUESTION 28 · REAL**City weather summary**

Concept: requests + JSON Source: s12.py

PROBLEM STATEMENT

Design a function that accepts a parsed weather response and returns a one-line city temperature summary.

REQUIREMENTS

Use the nested data[0] structure shown in the source; do not perform an actual network call for the exercise.

EXAMPLE INPUT

```
{"data":[{"temp":31.2,"city_name":"Mumbai"}]}
```

EXPECTED OUTPUT

```
Mumbai: 31.2 C
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
READ response["data"][0]
EXTRACT city_name and temp
FORMAT summary
```

QUESTION 29 · THINK**What should happen when an API returns bad JSON?**

Concept: requests + JSON Source: s12.py

PROBLEM STATEMENT

You call requests.get(url) and json.loads(response.text), but the server sometimes returns invalid text. Design the handling flow.

REQUIREMENTS

Describe the sequence for checking response data and catching parsing failures.

EXAMPLE INPUT

```
response.text = "not-json"
```

EXPECTED OUTPUT

```
Program should report a parsing problem without crashing
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
GET response -> validate expected response -> TRY json
```

QUESTION 30 · THINK**Choose the right file mode**

Concept: File modes Source: s12.py

PROBLEM STATEMENT

A report already exists and you need to add one new line without deleting existing lines. Which mode and why?

REQUIREMENTS

Compare w and a based on behavior.

EXAMPLE INPUT

```
Existing report + new line
```

EXPECTED OUTPUT

```
Use append mode a
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

```
OPEN in a -> WRITE new line -> CLOSE
```

CONCEPT NOTES

HTTP, JSON & FILE HANDLING - What to remember

requests.get()

WHAT YOU SHOULD KNOW

Performs an HTTP GET request and returns a response object.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of requests.get(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

response.text

WHAT YOU SHOULD KNOW

Provides response content as text, which can then be parsed when it contains JSON.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of response.text, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

json.loads()

WHAT YOU SHOULD KNOW

Converts JSON-formatted text into the corresponding Python data structure.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `json.loads()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

File modes

WHAT YOU SHOULD KNOW

The source uses `r`, `w` and `a` concepts for reading, writing and appending; closing the file releases the resource.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of File modes, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

API extraction

WHAT YOU SHOULD KNOW

Nested dictionaries/lists can be accessed after JSON parsing to read fields such as `data[0]["temp"]`.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of API extraction, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

API data extraction

WHAT YOU SHOULD KNOW

API data extraction is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of API data extraction, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

file modes

WHAT YOU SHOULD KNOW

file modes is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of file modes, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

open() / write() / close()

WHAT YOU SHOULD KNOW

open() / write() / close() is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of open() / write() / close(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

SESSION 3

JSON & WEB SCRAPING

JSON serialization, BeautifulSoup parsing, selectors, text extraction, cleaning and persistence

Source files: s13.py

SESSION FORMAT

15 questions in this session: 5 MCQs, 2 two-mark questions, 4 programming problems, 2 real-application problems and 2 programming-level thinking problems. Concept Notes follow the practice set.

QUESTION 31 · MCQ

BeautifulSoup parser

Concept: BeautifulSoup Source: s13.py

QUESTION

Which parser is used in the source when creating BeautifulSoup?

- A. json.parser
- B. html.parser
- C. csv.parser
- D. requests.parser

SKILL TESTED: BeautifulSoup

← BACK TO INDEX

PSEUDO-CODE / REASONING

HTML text -> BeautifulSoup(html, parser)

QUESTION 32 · MCQ

find_all()

Concept: BeautifulSoup selectors Source: s13.py

QUESTION

What does find_all() return in the source scraping example?

- A. A JSON string
- B. All matching tags
- C. Only the first tag
- D. A file handle

SKILL TESTED: BeautifulSoup selectors

← BACK TO INDEX

PSEUDO-CODE / REASONING

SEARCH DOM -> collect all matching tags

QUESTION 33 · MCQ

tag.text

Concept: BeautifulSoup text extraction Source: s13.py

QUESTION

What is the purpose of i.text in the source?

- A. HTML attribute name
- B. Text content of the tag
- C. CSS class
- D. HTTP status

SKILL TESTED: BeautifulSoup text extraction

← BACK TO INDEX

PSEUDO-CODE / REASONING

FOR each tag -> READ text content

QUESTION 34 · MCQ

Currency cleaning

Concept: String cleaning Source: s13.py

QUESTION

Why are replace("₹", "").replace(",", "") used in the price example?

- A. To encrypt the price
- B. To make it suitable for numeric conversion
- C. To sort HTML
- D. To create JSON

SKILL TESTED: String cleaning

← BACK TO INDEX

PSEUDO-CODE / REASONING

REMOVE currency symbol -> REMOVE comma -> CONVERT to float

QUESTION 35 · MCQ**Persist scraped titles**

Concept: Web scraping + file Source: s13.py

QUESTION

Which operation from the source writes each scraped title to a text file?

- A. f.read
- B. f.write
- C. json.loads
- D. find_all

SKILL TESTED: Web scraping + file

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

OPEN file -> WRITE formatted line

QUESTION 36 · 2MARK**Extract all titles**

Concept: BeautifulSoup Source: s13.py

PROBLEM STATEMENT

Write the line that extracts all matching product title tags from a BeautifulSoup object res when the HTML class is css-12xgt1.

REQUIREMENTS

Use find_all() exactly as demonstrated.

EXAMPLE INPUT

res is already a BeautifulSoup object

EXPECTED OUTPUT

```
title = res.find_all("div", {"class": "css-12xgt1"})
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

SEARCH div tags by class -> STORE matches

QUESTION 37 · 2MARK**Clean a price string**

Concept: String cleaning Source: s13.py

PROBLEM STATEMENT

Convert the text "₹17,500" into a numeric-compatible string using the two replace operations shown in the source.

REQUIREMENTS

Do not use regular expressions.

EXAMPLE INPUT

"₹17,500"

EXPECTED OUTPUT

"17500"

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

REMOVE ₹ -> REMOVE comma

QUESTION 38 · PROBLEM**Extract and print product titles**

Concept: BeautifulSoup Source: s13.py

PROBLEM STATEMENT

Given HTML containing repeated product title divs, parse it and print each title followed by a separator of 20 hyphens.

REQUIREMENTS

Use BeautifulSoup with html.parser and find_all().

EXAMPLE INPUT

```
<div class="title">Bag</div><div class="title">Shoes</div>
```

EXPECTED OUTPUT

```
Bag
-----
Shoes
-----
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

PARSE HTML -> FIND matching divs -> FOR each -> print

QUESTION 39 · PROBLEM**Filter prices from scraped text**

Concept: BeautifulSoup + string cleaning Source: s13.py

PROBLEM STATEMENT

Given matched price tags, print only prices between 17000 and 20000 inclusive.

REQUIREMENTS

For each tag, remove ₹ and commas, convert to float, then apply the numeric condition.

EXAMPLE INPUT

```
["₹16,999", "₹17,500", "₹20,000", "₹21,100"]
```

EXPECTED OUTPUT

```
₹17,500
₹20,000
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

FOR each price tag -> clean text -> float -> IF range

QUESTION 40 · PROBLEM**Save scraped data with numbering**

Concept: BeautifulSoup + file handling Source: s13.py

PROBLEM STATEMENT

Write the titles ["Bag","Shoes","Watch"] to demo.txt as 1.Bag, 2.Shoes, 3.Watch.

REQUIREMENTS

Use a counter and f.write().

EXAMPLE INPUT

```
["Bag", "Shoes", "Watch"]
```

EXPECTED OUTPUT

```
1.Bag
2.Shoes
3.Watch
```

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

OPEN file -> k=1 -> FOR item -> write k.item -> increm

QUESTION 41 • PROBLEM**Count matching products**

Concept: BeautifulSoup + filtering Source: s13.py

PROBLEM STATEMENT

Given simplified HTML product cards with title and price classes, count how many products fall within a target price range.

REQUIREMENTS

Parse the HTML, locate price tags, clean their text, convert to float, and count matches.

EXAMPLE INPUT

3 prices: ₹15000, ₹18500, ₹21000; range 17000-20000

EXPECTED OUTPUT

1 matching product

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

PARSE -> FIND prices -> CLEAN -> TEST range -> COUNT

QUESTION 42 • REAL**Price-monitor report**

Concept: BeautifulSoup + filtering + file Source: s13.py

PROBLEM STATEMENT

Build a daily report generator for a shopping site: find all product prices within a target range and save those original price strings to a text file.

REQUIREMENTS

Requirements: parse HTML, locate price tags, clean for numeric comparison, preserve original display text in report, write one match per line.

EXAMPLE INPUT

Prices: ₹12,999; ₹18,500; ₹20,000; target 17000-20000

EXPECTED OUTPUT

Report lines: ₹18,500 and ₹20,000

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

SCRAPE -> CLEAN for decision -> KEEP original text ->

QUESTION 43 • REAL**Product title export**

Concept: BeautifulSoup + file writing Source: s13.py

PROBLEM STATEMENT

Create a script that extracts all product titles from HTML and stores them in a numbered text report.

REQUIREMENTS

Requirements: use find_all(), use .text, number rows starting at 1, save with one item per line.

EXAMPLE INPUT

Two title divs: Bag, Shoes

EXPECTED OUTPUT

1. Bag
2. Shoes

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

PARSE -> FIND -> EXTRACT TEXT -> NUMBER -> WRITE

QUESTION 44 · THINK**Make the scraper resilient**

Concept: BeautifulSoup + data cleaning Source: s13.py

PROBLEM STATEMENT

A site changes the class name used for product titles. Explain how you would reduce hard-coding risk while staying within the concepts present in the source.

REQUIREMENTS

Discuss selector assumptions, validation, and graceful empty results.

EXAMPLE INPUT

Page has no matching title tags

EXPECTED OUTPUT

Return an empty result or a clear "No products found" message rather than crashing

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

PARSE -> SELECT -> CHECK matches -> PROCESS if present

QUESTION 45 · THINK**Separate raw and cleaned values**

Concept: String cleaning + scraping Source: s13.py

PROBLEM STATEMENT

Why is it useful to keep the original price string and separately compute a cleaned numeric value?

REQUIREMENTS

Explain the difference between display data and comparison data.

EXAMPLE INPUT

raw = "₹19,999"

EXPECTED OUTPUT

raw stays for reporting; numeric 19999 is used for comparisons

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

KEEP raw text -> DERIVE numeric value -> USE numeric f

CONCEPT NOTES

JSON & WEB SCRAPING - What to remember

BeautifulSoup

WHAT YOU SHOULD KNOW

Parses HTML so tags can be located and their text extracted.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of BeautifulSoup, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

find_all()

WHAT YOU SHOULD KNOW

Finds all tags matching the selector criteria shown in the source.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of find_all(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

tag.text

WHAT YOU SHOULD KNOW

Extracts the text content from a matched HTML tag.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `tag.text`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Price cleaning

WHAT YOU SHOULD KNOW

Replacing currency symbols and commas creates numeric-compatible text for comparison.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Price cleaning, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Scrape -> filter -> persist

WHAT YOU SHOULD KNOW

A practical pipeline: parse HTML, locate values, clean them, apply rules, and write results.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Scrape -> filter -> persist, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

json.dumps() / json.loads()

WHAT YOU SHOULD KNOW

`json.dumps()` / `json.loads()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `json.dumps()` / `json.loads()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

html.parser

WHAT YOU SHOULD KNOW

`html.parser` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `html.parser`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

replace() / numeric cleaning

WHAT YOU SHOULD KNOW

`replace()` / numeric cleaning is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `replace()` / numeric cleaning, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

writing scraped data

WHAT YOU SHOULD KNOW

writing scraped data is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of writing scraped data, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

SESSION 4

MODULES, TIME & REGEX VALIDATION

imports, aliases, sleep(), local time, regular expressions and validation logic

Source files: s10.py

SESSION FORMAT

15 questions in this session: 5 MCQs, 2 two-mark questions, 4 programming problems, 2 real-application problems and 2 programming-level thinking problems. Concept Notes follow the practice set.

QUESTION 46 • MCQ

Import alias

Concept: Modules Source: s10.py

QUESTION

What does "from mymodule import hi_msg as hm" allow?

- A. Imports every module
- B. Imports hi_msg under the local name hm
- C. Imports only time
- D. Creates a package

SKILL TESTED: Modules

← BACK TO INDEX

PSEUDO-CODE / REASONING

IMPORT function -> GIVE local alias hm -> CALL hm()

QUESTION 47 • MCQ

sleep()

Concept: time Source: s10.py

QUESTION

What does sleep(5) do in the password retry example?

- A. Repeats code 5 times
- B. Pauses execution about 5 seconds
- C. Ends program
- D. Waits for 5 passwords

SKILL TESTED: time

← BACK TO INDEX

PSEUDO-CODE / REASONING

CALL sleep(5) -> PAUSE -> CONTINUE

QUESTION 48 · MCQ**Regex character class**

Concept: re.search Source: s10.py

QUESTION

What does [A-Z] in the password regex check?

- A. Any digit
- B. Any lowercase letter
- C. Any uppercase letter
- D. Any punctuation

SKILL TESTED: re.search

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

SEARCH for any character in A-Z

QUESTION 49 · MCQ**Retry counter**

Concept: Loop logic Source: s10.py

QUESTION

In the source password retry example, what happens when i == 3?

- A. Program exits
- B. Password becomes valid
- C. Sleep and reset i
- D. Imports a module

SKILL TESTED: Loop logic

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

IF attempts hit 3 -> SLEEP -> RESET i

QUESTION 50 · MCQ**localtime()**

Concept: time.localtime Source: s10.py

QUESTION

What does time.localtime() provide in the source?

- A. Random number
- B. Local time information
- C. Network response
- D. Regex match

SKILL TESTED: time.localtime

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

CALL localtime -> RECEIVE local time structure

QUESTION 51 · 2MARK**Alias an imported function**

Concept: import syntax Source: s10.py

PROBLEM STATEMENT

Write the import statement that lets the program call hi_msg() as hm().

REQUIREMENTS

Use from ... import ... as syntax.

EXAMPLE INPUT

mymodule contains hi_msg

EXPECTED OUTPUT

from mymodule import hi_msg as hm

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

IMPORT hi_msg -> ALIAS as hm

QUESTION 52 • 2MARK**One regex requirement**

Concept: Regex Source: s10.py

PROBLEM STATEMENT

Write a regex search condition that checks whether a password contains at least one digit.

REQUIREMENTS

Use re.search with the character class shown by the source pattern.

EXAMPLE INPUT

```
pwd is a string
```

EXPECTED OUTPUT

```
re.search(r"[0-9]", pwd)
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

SEARCH for a character in 0-9

QUESTION 53 • PROBLEM**Password rule validator**

Concept: regex + control flow Source: s10.py

PROBLEM STATEMENT

Write a function that validates a password using the same five checks as the source: length rule, uppercase, lowercase, digit, and one special character from @#\$%^&*.

REQUIREMENTS

Return "Valid password!!!" when all checks pass, otherwise "Invalid password!!!". Keep the checks independent.

EXAMPLE INPUT

```
"Abc123@x"
```

EXPECTED OUTPUT

```
Valid password!!!
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

CHECK length -> CHECK uppercase -> lowercase -> digit

QUESTION 54 • PROBLEM**Retry with sleep mode**

Concept: while + sleep Source: s10.py

PROBLEM STATEMENT

Create a retry loop that accepts a password, gives three attempts, then waits 2 seconds before resetting the attempt count.

REQUIREMENTS

Use while True, an attempt counter, break on success, and sleep on the third failed attempt.

EXAMPLE INPUT

```
Expected success on fourth entered attempt
```

EXPECTED OUTPUT

```
Try again... / Sleep mode... / Hey, Welcome!!!
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

LOOP forever -> INPUT -> IF correct BREAK -> ELSE incr

QUESTION 55 · PROBLEM**Format local time**

Concept: time.localtime Source: s10.py

PROBLEM STATEMENT

Use localtime() to format the current local hour, minute and second as HH:MM:SS.

REQUIREMENTS

Use the tuple indexes shown by the source and zero-pad each part.

EXAMPLE INPUT

Current local time structure

EXPECTED OUTPUT

For example: 08:04:09

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

GET localtime -> READ hour/minute/second -> FORMAT two

QUESTION 56 · PROBLEM**Use a user-defined module cleanly**

Concept: modules + alias Source: s10.py

PROBLEM STATEMENT

Suppose mymodule.py exposes hello_msg(name) and hi_msg(name). Write a main script that imports both and calls hi_msg through the alias hm.

REQUIREMENTS

Use an explicit import rather than import *.

EXAMPLE INPUT

mymodule has hi_msg and hello_msg

EXPECTED OUTPUT

Hello from the called functions

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

IMPORT needed function -> ALIAS -> CALL

QUESTION 57 · REAL**Account lockout guard**

Concept: regex + retries + sleep Source: s10.py

PROBLEM STATEMENT

Design a login guard that checks a password format before accepting it and temporarily pauses after repeated failed entries.

REQUIREMENTS

Requirements: validate password shape with regex, count failed attempts, sleep after three failures, reset counter after sleep, stop when valid input is accepted.

EXAMPLE INPUT

Example: invalid passwords three times, then valid password

EXPECTED OUTPUT

Three failures -> Sleep mode -> next valid attempt is accepted

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

VALIDATE format -> TRACK failures -> SLEEP at threshol

QUESTION 58 · REAL**Reusable time utility module**

Concept: modules + time Source: s10.py

PROBLEM STATEMENT

Create a utility function in a module that returns the current local time as HH:MM:SS, then import it with an alias in another script.

REQUIREMENTS

Use time.localtime() inside the module and from ... import ... as in the main script.

EXAMPLE INPUT

No user input required

EXPECTED OUTPUT

Example: 20:43:15

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

MODULE -> DEFINE get_time() -> IMPORT AS alias -> CALL

QUESTION 59 · THINK**Design the regex checks**

Concept: regex validation Source: s10.py

PROBLEM STATEMENT

A password can fail multiple requirements at once. Explain why checking each regex condition separately makes the program easier to understand than one giant regular expression.

REQUIREMENTS

Compare independent validation with a single complex pattern.

EXAMPLE INPUT

Password fails uppercase and digit rules

EXPECTED OUTPUT

Independent checks identify each missing requirement and are easier to debug

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

CHECK each rule -> FAIL FAST or COLLECT issues -> REPORT

QUESTION 60 · THINK**Module import choice**

Concept: import strategies Source: s10.py

PROBLEM STATEMENT

When a module has two functions but the main script uses only one, explain the advantage of importing only that function versus using import *.

REQUIREMENTS

Focus on clarity and namespace control.

EXAMPLE INPUT

mymodule.py exports hello_msg and hi_msg

EXPECTED OUTPUT

Explicit import makes the dependency clear and avoids wildcard names

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

CHOOSE minimal dependency -> IMPORT explicit symbol

CONCEPT NOTES

MODULES, TIME & REGEX VALIDATION - What to remember

Modules / imports

WHAT YOU SHOULD KNOW

Imports make code reusable and allow functions to be called from another file.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Modules / imports, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Aliases

WHAT YOU SHOULD KNOW

from module import name as alias gives a shorter local name.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Aliases, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

time / sleep()

WHAT YOU SHOULD KNOW

time.localtime() exposes local time fields; sleep() pauses execution.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `time / sleep()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Regex validation

WHAT YOU SHOULD KNOW

`re.search()` with character classes can test whether required characters appear in a string.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Regex validation, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Retry logic

WHAT YOU SHOULD KNOW

A loop plus an attempt counter can implement repeated attempts and a temporary sleep/lockout.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of Retry logic, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

import

WHAT YOU SHOULD KNOW

`import` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of import, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

from ... import

WHAT YOU SHOULD KNOW

from ... import is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of from ... import, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

aliases

WHAT YOU SHOULD KNOW

aliases is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of aliases, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

time.localtime()

WHAT YOU SHOULD KNOW

`time.localtime()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `time.localtime()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

re.search()

WHAT YOU SHOULD KNOW

`re.search()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `re.search()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

password validation

WHAT YOU SHOULD KNOW

password validation is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of password validation, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

retry / lockout logic

WHAT YOU SHOULD KNOW

retry / lockout logic is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of retry / lockout logic, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

SESSION 5

PANDAS & CSV DATA ANALYSIS

Path, read_csv(), inspection, missing data, derived columns, filtering, duplicates and export

Source files: s14.py, marks.csv

SESSION FORMAT

15 questions in this session: 5 MCQs, 2 two-mark questions, 4 programming problems, 2 real-application problems and 2 programming-level thinking problems. Concept Notes follow the practice set.

QUESTION 61 • MCQ

Path(__file__).parent

Concept: pathlib Source: s14.py

QUESTION

What is csv_file = Path(__file__).parent / "marks.csv" doing?

- A. Reads CSV immediately
- B. Builds a path relative to the script file
- C. Deletes the CSV
- D. Converts CSV to JSON

SKILL TESTED: pathlib

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

GET script directory -> APPEND marks.csv

QUESTION 62 • MCQ

head() / tail()

Concept: pandas inspection Source: s14.py

QUESTION

What does data.head(2) do in the source?

- A. Last 2 rows
- B. First 2 rows
- C. First 2 columns
- D. Duplicate rows

SKILL TESTED: pandas inspection

PSEUDO-CODE / REASONING

TAKE dataframe -> SELECT first 2 rows

[← BACK TO INDEX](#)**QUESTION 63 · MCQ****fillna vs dropna**

Concept: pandas missing data Source: s14.py

QUESTION

Which operation in the source fills missing values with 0?

- A. dropna
- B. fillna(0)
- C. duplicated
- D. iloc

SKILL TESTED: pandas missing data

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

MISSING -> REPLACE with 0

QUESTION 64 · MCQ**loc vs iloc**

Concept: pandas indexing Source: s14.py

QUESTION

Which slice includes row label 6 with loc[2:6] but excludes position 6 with iloc[2:6]?

- A. Both exclude 6
- B. loc includes 6, iloc excludes position 6
- C. iloc includes 6, loc excludes 6
- D. They are identical

SKILL TESTED: pandas indexing

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

LOC -> LABELS / inclusive end

ILOC -> POSITIONS / exclusive end

QUESTION 65 · MCQ**apply(lambda)**

Concept: pandas derived column Source: s14.py

QUESTION

Why is data["Result"] = data["Total"].apply(lambda n: "pass" if n > 200 else "fail") used?

- A. Sort rows
- B. Create a new derived result column
- C. Remove duplicates
- D. Read another CSV

SKILL TESTED: pandas derived column

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

FOR each Total -> apply condition -> STORE Result

QUESTION 66 · 2MARK**Create Total column**

Concept: pandas arithmetic Source: s14.py

PROBLEM STATEMENT

Write the pandas statement that creates Total from Physics, Maths and Chemistry.

REQUIREMENTS

Use the exact column names shown in the source.

EXAMPLE INPUT

data has Physics, Maths, Chemistry

EXPECTED OUTPUT

data["Total"] = data["Physics"] + data["Maths"] + data["Chemistry"]

PSEUDO-CODE / REASONING

READ three columns -> ADD row-wise -> ASSIGN Total

[← BACK TO INDEX](#)**QUESTION 67 · 2MARK****Difference between loc and iloc**

Concept: pandas indexing Source: s14.py

PROBLEM STATEMENT

In one or two sentences, explain the difference between `data.loc[2:6]` and `data.iloc[2:6]` based on the source notes.

REQUIREMENTS

Mention labels vs positions and the end boundary behavior.

EXAMPLE INPUT

```
data.loc[2:6] / data.iloc[2:6]
```

EXPECTED OUTPUT

```
loc uses labels and includes the end label; iloc uses positions and follows slicing rules
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

LOC = labels; ILOC = integer positions

QUESTION 68 · PROBLEM**Clean and classify marks**

Concept: pandas + missing data + apply Source: s14.py

PROBLEM STATEMENT

Given a DataFrame with Maths, Physics and Chemistry, replace missing values with 0, compute Total, then create Result as pass if Total > 200 else fail.

REQUIREMENTS

Use `fillna(0)`, a derived Total column, and `apply(lambda)`.

EXAMPLE INPUT

```
Rows: [Maths=70, Physics=80, Chemistry=60] and [Maths=None, Physics=90, Chemistry=50]
```

EXPECTED OUTPUT

```
Totals 210 and 140; Results pass and fail
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

FILL missing -> COMPUTE Total -> APPLY pass/fail rule

QUESTION 69 · PROBLEM**Filter failing Maths rows**

Concept: pandas loc Source: s14.py

PROBLEM STATEMENT

Using a DataFrame with a Result column, select rows where Maths > 30 AND Result == "fail".

REQUIREMENTS

Use `.loc` with both boolean conditions combined using `&`.

EXAMPLE INPUT

```
Maths=[25,40,35], Result=[fail,fail,pass]
```

EXPECTED OUTPUT

```
Rows where Maths > 30 and Result is fail -> the second row
```

[← BACK TO INDEX](#)**PSEUDO-CODE / REASONING**

BUILD condition1 -> condition2 -> COMBINE with & -> LOC

QUESTION 70 • PROBLEM**Remove duplicate records**

Concept: pandas duplicates Source: s14.py

PROBLEM STATEMENT

Given a DataFrame with duplicate student rows, detect and remove duplicates.

REQUIREMENTS

Show both the duplicated() check and drop_duplicates() usage.

EXAMPLE INPUT

Rows: A, B, B

EXPECTED OUTPUT

duplicated() identifies the repeated B row; drop_duplicates() leaves A and B

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

CHECK duplicates -> REPORT -> DROP duplicates

QUESTION 71 • PROBLEM**Export cleaned results**

Concept: pandas to_csv Source: s14.py

PROBLEM STATEMENT

After cleaning and deriving Result, write the updated DataFrame to a new CSV file named new.csv.

REQUIREMENTS

Use to_csv(); do not overwrite the original file in the exercise.

EXAMPLE INPUT

DataFrame after transformations

EXPECTED OUTPUT

new.csv is created with the updated columns

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

TRANSFORM dataframe -> EXPORT to new.csv

QUESTION 72 • REAL**Student performance report**

Concept: pandas + CSV Source: s14.py + marks.csv

PROBLEM STATEMENT

Create a reusable analysis flow for a class marks CSV: inspect the first rows, fill missing marks with 0, compute Total and Result, then export a cleaned CSV.

REQUIREMENTS

Requirements: read via Path-based path, inspect with head(), clean, derive columns, save to a new file.

EXAMPLE INPUT

Sample rows with Maths/Physics/Chemistry

EXPECTED OUTPUT

Cleaned CSV with Total and Result columns

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

READ CSV -> INSPECT -> CLEAN -> DERIVE -> EXPORT

QUESTION 73 · REAL**Academic failure filter**

Concept: pandas loc + derived column Source: s14.py

PROBLEM STATEMENT

From a cleaned marks dataset, create a subset for students who have Maths > 30 but still have a failing overall Result.

REQUIREMENTS

Use .loc and combine conditions with &.

EXAMPLE INPUT

Maths and Result columns present

EXPECTED OUTPUT

Filtered DataFrame contains only rows meeting both conditions

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

READ dataframe -> BUILD boolean mask -> LOC subset

QUESTION 74 · THINK**Fill or drop missing data?**

Concept: fillna / dropna Source: s14.py + marks.csv

PROBLEM STATEMENT

A marks dataset has 5% missing values and each row represents a student. Explain when fillna(0) is appropriate and when dropna() may be safer.

REQUIREMENTS

Compare the meaning of missing data in a real academic dataset.

EXAMPLE INPUT

Missing marks in some rows

EXPECTED OUTPUT

Choose based on what missing means; do not treat missing as zero automatically without business meaning

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

UNDERSTAND missingness -> CHOOSE fill or drop -> VALIDATE

QUESTION 75 · THINK**Vectorized column calculation vs row loop**

Concept: pandas Source: s14.py

PROBLEM STATEMENT

Explain why the source uses column arithmetic and apply() instead of looping through every row manually.

REQUIREMENTS

Focus on readability and dataframe operations.

EXAMPLE INPUT

Physics + Maths + Chemistry columns

EXPECTED OUTPUT

Prefer vectorized column arithmetic for direct numeric calculations; use apply when row-level function logic is needed

[← BACK TO INDEX](#)

PSEUDO-CODE / REASONING

USE column operations for vector math -> APPLY only for

CONCEPT NOTES

PANDAS & CSV DATA ANALYSIS - What to remember

Path

WHAT YOU SHOULD KNOW

`Path(__file__).parent / filename` builds a filesystem path relative to the script.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `Path`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

read_csv()

WHAT YOU SHOULD KNOW

Loads CSV data into a pandas DataFrame.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `read_csv()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

head() / tail() / columns

WHAT YOU SHOULD KNOW

Quick inspection tools for understanding dataframe structure.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `head()` / `tail()` / `columns`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

fillna() / dropna()

WHAT YOU SHOULD KNOW

Different strategies for missing data: fill values or remove rows with missing values.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `fillna()` / `dropna()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

Derived columns / apply()

WHAT YOU SHOULD KNOW

Create new columns from existing data, using vectorized arithmetic or element-wise functions.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `Derived columns / apply()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

loc / iloc

WHAT YOU SHOULD KNOW

`loc` selects by labels; `iloc` selects by integer positions.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `loc` / `iloc`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

duplicates / export

WHAT YOU SHOULD KNOW

`duplicated()` detects repeats, `drop_duplicates()` removes them, and `to_csv()` exports the result.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `duplicates` / `export`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

pathlib.Path

WHAT YOU SHOULD KNOW

`pathlib.Path` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `pathlib.Path`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

pandas.read_csv()

WHAT YOU SHOULD KNOW

`pandas.read_csv()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `pandas.read_csv()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

head() / tail()**WHAT YOU SHOULD KNOW**

`head()` / `tail()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `head()` / `tail()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

columns**WHAT YOU SHOULD KNOW**

`columns` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `columns`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

derived columns

WHAT YOU SHOULD KNOW

derived columns is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of derived columns, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

apply(lambda)

WHAT YOU SHOULD KNOW

apply(lambda) is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of apply(lambda), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

duplicated() / drop_duplicates()

WHAT YOU SHOULD KNOW

duplicated() / drop_duplicates() is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of duplicated() / drop_duplicates(), show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

to_csv()**WHAT YOU SHOULD KNOW**

`to_csv()` is used in the supplied material for practical Python work in this session. Review its role, syntax pattern, and where it fits in the workflow shown by the source examples.

IMPORTANT RULES

Use the source pattern for the intended job; do not use a transformation function when you need selection, and do not use a selection function when you need transformation.

COMMON MISTAKES

Confusing text data with parsed data, forgetting to convert iterators to lists for display, using the wrong file mode, or mixing data-cleaning decisions with presentation logic.

INTERVIEW / CLASS TEST POINTS

Be ready to explain the role of `to_csv()`, show a short example, and distinguish it from the closest alternative in the source.

PROBLEM-SOLVING TIP

Start by naming the input, the transformation/selection step, and the expected output. Then choose the smallest source-backed tool that solves that exact step.

MINI CHECK

1) What is the input type? 2) What does the operation return? 3) What edge case could break the logic?

SOURCE COVERAGE MATRIX

Every attached source is represented

The five-session structure groups the six supplied files without introducing unrelated syllabus content.

S1 FUNCTIONAL PYTHON TOOLS	s11.py	List comprehension Conditional / nested ternary expression lambda functions filter() map() map/filter pipeline
S2 HTTP, JSON & FILE HANDLING	s12.py	requests.get() response.text json.loads() API data extraction file modes open() / write() / close()
S3 JSON & WEB SCRAPING	s13.py	json.dumps() / json.loads() BeautifulSoup html.parser find_all() tag.text replace() / numeric cleaning writing scraped data
S4 MODULES, TIME & REGEX VALIDATION	s10.py	import from ... import aliases time / sleep() time.localtime() re.search() password validation retry / lockout logic
S5 PANDAS & CSV DATA ANALYSIS	s14.py marks.csv	pathlib.Path pandas.read_csv() head() / tail() columns fillna() / dropna() derived columns apply(lambda) loc / iloc duplicated() / drop_duplicates() to_csv()

QUESTION-TO-SOURCE MAPPING

1	MCQ	s11.py	List comprehension
2	MCQ	s11.py	Conditional expression
3	MCQ	s11.py	lambda
4	MCQ	s11.py	filter()
5	MCQ	s11.py	map() / filter()
6	2MARK	s11.py	List comprehension
7	2MARK	s11.py	map() + filter()
8	PROBLEM	s11.py	map() + filter()
9	PROBLEM	s11.py	List comprehension + ternary
10	PROBLEM	s11.py	lambda + filter + map
11	PROBLEM	s11.py	map/filter design
12	REAL	s11.py	filter + map
13	REAL	s11.py	lambda + map
14	THINK	s11.py	map vs filter
15	THINK	s11.py	filter + map
16	MCQ	s12.py	requests
17	MCQ	s12.py	JSON conversion
18	MCQ	s12.py	File handling
19	MCQ	s12.py	requests + map
20	MCQ	s12.py	File handling
21	2MARK	s12.py	json.loads()
22	2MARK	s12.py	File modes
23	PROBLEM	s12.py	requests + JSON + map
24	PROBLEM	s12.py	requests + file writing
25	PROBLEM	s12.py	requests + JSON
26	PROBLEM	s12.py	requests + JSON + map
27	REAL	s12.py	requests + JSON + map + file

28	REAL	s12.py	requests + JSON
29	THINK	s12.py	requests + JSON
30	THINK	s12.py	File modes
31	MCQ	s13.py	BeautifulSoup
32	MCQ	s13.py	BeautifulSoup selectors
33	MCQ	s13.py	BeautifulSoup text extraction
34	MCQ	s13.py	String cleaning
35	MCQ	s13.py	Web scraping + file
36	2MARK	s13.py	BeautifulSoup
37	2MARK	s13.py	String cleaning
38	PROBLEM	s13.py	BeautifulSoup
39	PROBLEM	s13.py	BeautifulSoup + string cleaning
40	PROBLEM	s13.py	BeautifulSoup + file handling
41	PROBLEM	s13.py	BeautifulSoup + filtering
42	REAL	s13.py	BeautifulSoup + filtering + file
43	REAL	s13.py	BeautifulSoup + file writing
44	THINK	s13.py	BeautifulSoup + data cleaning
45	THINK	s13.py	String cleaning + scraping
46	MCQ	s10.py	Modules
47	MCQ	s10.py	time
48	MCQ	s10.py	re.search
49	MCQ	s10.py	Loop logic
50	MCQ	s10.py	time.localtime
51	2MARK	s10.py	import syntax
52	2MARK	s10.py	Regex
53	PROBLEM	s10.py	regex + control flow
54	PROBLEM	s10.py	while + sleep
55	PROBLEM	s10.py	time.localtime
56	PROBLEM	s10.py	modules + alias
57	REAL	s10.py	regex + retries + sleep
58	REAL	s10.py	modules + time
59	THINK	s10.py	regex validation
60	THINK	s10.py	import strategies
61	MCQ	s14.py	pathlib
62	MCQ	s14.py	pandas inspection
63	MCQ	s14.py	pandas missing data
64	MCQ	s14.py	pandas indexing
65	MCQ	s14.py	pandas derived column
66	2MARK	s14.py	pandas arithmetic
67	2MARK	s14.py	pandas indexing
68	PROBLEM	s14.py	pandas + missing data + apply
69	PROBLEM	s14.py	pandas loc
70	PROBLEM	s14.py	pandas duplicates
71	PROBLEM	s14.py	pandas to_csv
72	REAL	s14.py + marks.csv	pandas + CSV
73	REAL	s14.py	pandas loc + derived column
74	THINK	s14.py + marks.csv	fillna / dropna
75	THINK	s14.py	pandas

ANSWER KEY & EXPLANATIONS

Complete answers for all 75 questions

Answers are grouped by session to mirror the practice section. The question section remains solution-free.

FUNCTIONAL PYTHON TOOLS

ANSWER 1 - List comprehension output

Source: s11.py Concept: List comprehension

CORRECT ANSWER: B

The comprehension applies $x * 2$ to every item in the original list.

← BACK TO INDEX

ANSWER 2 - Ternary selection

Source: s11.py Concept: Conditional expression

CORRECT ANSWER: B

$17 \geq 18$ is false, so the else expression is selected.

← BACK TO INDEX

ANSWER 3 - lambda expression

Source: s11.py Concept: lambda

CORRECT ANSWER: C

The lambda formats the supplied name into the string Hello Dev.

← BACK TO INDEX

ANSWER 4 - filter() result

Source: s11.py Concept: filter()

CORRECT ANSWER: B

Only 20 and 21 satisfy the predicate $n \geq 20$.

← BACK TO INDEX

ANSWER 5 - map vs filter

Source: s11.py Concept: map() / filter()

CORRECT ANSWER: C

filter controls membership; map transforms each input item.

← BACK TO INDEX

ANSWER 6 - Write a conditional list comprehension

Source: s11.py Concept: List comprehension

SOLUTION

```
[n if n % 2 == 0 else n ** 2 for n in [1, 2, 3, 4]]
```

EXPLANATION

Each number is tested for evenness; odd numbers are squared.

← BACK TO INDEX

ANSWER 7 - Explain the final list

Source: s11.py Concept: map() + filter()

SOLUTION

```
list(map(lambda m: m**2, filter(lambda a: a >= 20, [3,35,62,2,72,18,9])))
```

EXPLANATION

filter keeps 35, 62 and 72. map squares those three values.

← BACK TO INDEX

ANSWER 8 - Transform qualifying scores

Source: s11.py Concept: map() + filter()

SOLUTION

```
scores = [42, 55, 80, 37, 91]
result = list(map(lambda x: f"Pass: {x}%", filter(lambda x: x >= 50, scores)))
print(result)
```

EXPLANATION

The filter removes scores below 50. map formats each remaining score as a percentage label.

COMPLEXITY

Time: O(n) | Space: O(n) for the resulting list

← BACK TO INDEX

ANSWER 9 - Create a conditional transformed list

Source: s11.py Concept: List comprehension + ternary

SOLUTION

```
values = [3,5,6,2,7,8,9]
result = [x if x % 2 == 0 else x ** 2 for x in values]
print(result)
```

EXPLANATION

The expression applies one of two transformations for each item.

← BACK TO INDEX

ANSWER 10 - Build a reusable threshold pipeline

Source: s11.py Concept: lambda + filter + map

SOLUTION

```
def squared_at_least(values, threshold):
    return list(map(lambda x: x ** 2, filter(lambda x: x >= threshold, values)))

print(squared_at_least([10,25,30,15], 20))
```

EXPLANATION

The function composes two single-purpose steps in a readable pipeline.

← BACK TO INDEX

ANSWER 11 - Choose map or filter correctly

Source: s11.py Concept: map/filter design

SOLUTION

```
qty = [0,2,5,1]
result = list(map(lambda x: f"{x} units", filter(lambda x: x > 0, qty)))
print(result)
```

EXPLANATION

filter removes zero stock; map converts each remaining number to a readable label.

← BACK TO INDEX

ANSWER 12 - Customer price eligibility report

Source: s11.py Concept: filter + map

SOLUTION

```
prices = [499, 1200, 999, 2500]
for line in map(lambda x: f"Eligible: ₹{x}", filter(lambda x: x >= 1000, prices)):
    print(line)
```

EXPLANATION

The pipeline separates selection from presentation.

← BACK TO INDEX

ANSWER 13 - Attendance status generator

Source: s11.py Concept: lambda + map

SOLUTION

```
attendance = [82, 68, 91, 74]
status = list(map(lambda x: "Present" if x >= 75 else "Short", attendance))
print(status)
```

EXPLANATION

map applies the same conditional transformation to every value.

← BACK TO INDEX

ANSWER 14 - Which tool should you choose?

Source: s11.py Concept: map vs filter

SOLUTION

```
# Reasoned approach
SELECT -> FILTER >= 18
TRANSFORM -> MAP square
RETURN [324,484]
```

Conclusion:

EXPLANATION

The solution follows the requirements and source concept.

LEETCODE SKILL

Pattern: filter then transform

← BACK TO INDEX

ANSWER 15 - Debug the pipeline

Source: s11.py Concept: filter + map

SOLUTION

```
list(map(lambda x: x ** 2, [1,2,3]))
```

EXPLANATION

The function passed to filter is interpreted as a truth-test; it does not create transformed output.

← BACK TO INDEX

HTTP, JSON & FILE HANDLING

ANSWER 16 - requests.get() content

Source: s12.py Concept: requests

CORRECT ANSWER: B

The examples use response.text as a string before passing it to json.loads().

← BACK TO INDEX

ANSWER 17 - json.loads()

Source: s12.py Concept: JSON conversion

CORRECT ANSWER: B

json.loads parses a JSON-formatted string into a Python value.

← BACK TO INDEX

ANSWER 18 - File mode

Source: s12.py Concept: File handling

CORRECT ANSWER: C

The source explicitly uses w for writing the file.

← BACK TO INDEX

ANSWER 19 - API titles with map

Source: s12.py Concept: requests + map

CORRECT ANSWER: B

map transforms each product dictionary into its title value.

← BACK TO INDEX

ANSWER 20 - Reading after write

Source: s12.py Concept: File handling

CORRECT ANSWER: C

close finishes the file operation and releases the resource.

← BACK TO INDEX

ANSWER 21 - Parse a JSON string

Source: s12.py Concept: json.loads()

SOLUTION

```
json.loads(text)
```

EXPLANATION

json.loads parses the JSON string into a Python list containing a dictionary.

← BACK TO INDEX

ANSWER 22 - List the file modes

Source: s12.py Concept: File modes

SOLUTION

See the worked solution.

EXPLANATION

The source notes t, r, w, a and describes the primary read/write/append roles.

← BACK TO INDEX

ANSWER 23 - Extract API product titles

Source: s12.py Concept: requests + JSON + map

SOLUTION

```
import json

plain_text = '[{"title": "Bag"}, {"title": "Shoes"}, {"title": "Watch"}]'
res = json.loads(plain_text)
titles = list(map(lambda item: item["title"], res))
print(titles)
```

EXPLANATION

The source demonstrates exactly this pattern with product dictionaries returned from an API.

← BACK TO INDEX

ANSWER 24 - Write API data to a text file

Source: s12.py Concept: requests + file writing

SOLUTION

```
titles = ["Bag", "Shoes", "Watch"]
f = open("demo.txt", "w")
for title in titles:
    f.write(f"{title}\n")
f.close()
```

EXPLANATION

This mirrors the source file-writing loop.

← BACK TO INDEX

ANSWER 25 - Read a simple API temperature

Source: s12.py Concept: requests + JSON

SOLUTION

```
import json
text = '{"data":[{"temp":28.4}]}'
res = json.loads(text)
print(res["data"][0]["temp"])
```

EXPLANATION

The source weather example follows the same nested access pattern.

← BACK TO INDEX

ANSWER 26 - Create a small API report

Source: s12.py Concept: requests + JSON + map

SOLUTION

```
data = [{"title": "Bag", "price": 50}, {"title": "Pen", "price": 10}]
report = list(map(lambda x: f"{x['title']} - {x['price']}", data))
print(report)
```

EXPLANATION

map is used to transform each API record into a presentation string.

← BACK TO INDEX

ANSWER 27 - Product API reporting job

Source: s12.py Concept: requests + JSON + map + file

SOLUTION

```
import json
text = '[{"title":"Bag","price":50},{\"title\":\"Shoes\",\"price\":80}]'
products = json.loads(text)
with open("report.txt", "w") as f:
    for item in products:
        f.write(f"{item[\"title\"]} - {item[\"price\"]}\n")
```

EXPLANATION

This combines the API/JSON extraction and file-writing patterns shown in the source.

← BACK TO INDEX

ANSWER 28 - City weather summary

Source: s12.py Concept: requests + JSON

SOLUTION

```
def weather_summary(res):
    item = res["data"][0]
    return f"{item[\"city_name\"]}: {item[\"temp\"]} C"

print(weather_summary({"data":[{"temp":31.2,\"city_name\":\"Mumbai\"}]}))
```

EXPLANATION

The exercise reuses the nested JSON structure in the source without introducing unrelated API behavior.

← BACK TO INDEX

ANSWER 29 - What should happen when an API returns bad JSON?

Source: s12.py Concept: requests + JSON

SOLUTION

```
# Reasoned approach
GET response -> validate expected response -> TRY json.loads -> EXCEPT parse failure

Conclusion:
```

EXPLANATION

The solution follows the requirements and source concept.

INTERVIEW NOTE

Pattern: separate network errors from parsing errors

← BACK TO INDEX

ANSWER 30 - Choose the right file mode

Source: s12.py Concept: File modes

SOLUTION

```
# Reasoned approach
OPEN in a -> WRITE new line -> CLOSE

Conclusion:
```

EXPLANATION

w truncates/recreates the file; a appends to existing content.

← BACK TO INDEX

JSON & WEB SCRAPING

ANSWER 31 - BeautifulSoup parser

Source: s13.py Concept: BeautifulSoup

CORRECT ANSWER: B

The source uses html.parser.

← BACK TO INDEX

ANSWER 32 - find_all()

Source: s13.py Concept: BeautifulSoup selectors

CORRECT ANSWER: B

find_all returns all matching elements for the provided selector criteria.

← BACK TO INDEX

ANSWER 33 - tag.text

Source: s13.py Concept: BeautifulSoup text extraction

CORRECT ANSWER: B

The source prints i.text to get the content inside each matched tag.

← BACK TO INDEX

ANSWER 34 - Currency cleaning

Source: s13.py Concept: String cleaning

CORRECT ANSWER: B

Removing the symbol and comma makes the text directly convertible to a numeric value.

← BACK TO INDEX

ANSWER 35 - Persist scraped titles

Source: s13.py Concept: Web scraping + file

CORRECT ANSWER: B

The source writes each scraped title with f.write().

← BACK TO INDEX

ANSWER 36 - Extract all titles

Source: s13.py Concept: BeautifulSoup

SOLUTION

```
title = res.find_all("div", {"class": "css-12xgt1"})
```

EXPLANATION

This follows the selector pattern in the source.

← BACK TO INDEX

ANSWER 37 - Clean a price string

Source: s13.py Concept: String cleaning

SOLUTION

```
value = "₹17,500".replace("₹", "").replace(",", "")
```

EXPLANATION

The source uses exactly these replacements before calling float().

← BACK TO INDEX

ANSWER 38 - Extract and print product titles

Source: s13.py Concept: BeautifulSoup

SOLUTION

```
from bs4 import BeautifulSoup

html = '<div class="title">Bag</div><div class="title">Shoes</div>'
res = BeautifulSoup(html, "html.parser")
title = res.find_all("div", {"class": "title"})
for item in title:
    print(item.text)
    print("-" * 20)
```

EXPLANATION

This reproduces the source pattern of find_all(), .text and a separator.

← BACK TO INDEX

ANSWER 39 - Filter prices from scraped text

Source: s13.py Concept: BeautifulSoup + string cleaning

SOLUTION

```
prices = ["₹16,999", "₹17,500", "₹20,000", "₹21,100"]
for text in prices:
    value = float(text.replace("₹", "").replace(",", ""))
    if 17000 <= value <= 20000:
        print(text)
```

EXPLANATION

The source applies the same cleaning and range comparison to scraped price text.

← BACK TO INDEX

ANSWER 40 - Save scraped data with numbering

Source: s13.py Concept: BeautifulSoup + file handling

SOLUTION

```
titles = ["Bag", "Shoes", "Watch"]
with open("demo.txt", "w") as f:
    k = 1
    for title in titles:
        f.write(f"{k}.{title}\n")
        k += 1
```

EXPLANATION

This mirrors the source counter + file-writing pattern.

← BACK TO INDEX

ANSWER 41 - Count matching products

Source: s13.py Concept: BeautifulSoup + filtering

SOLUTION

```
from bs4 import BeautifulSoup

html = (
    '<h3 class="price">₹15,000</h3>'
    '<h3 class="price">₹18,500</h3>'
    '<h3 class="price">₹21,000</h3>'
)
res = BeautifulSoup(html, "html.parser")
prices = res.find_all("h3", {"class": "price"})
count = 0
for p in prices:
    value = float(p.text.replace("₹", "").replace(",", ""))
    if 17000 <= value <= 20000:
        count += 1
print(count)
```

EXPLANATION

The solution follows the same scrape-clean-filter pipeline from the source.

← BACK TO INDEX

ANSWER 42 - Price-monitor report

Source: s13.py Concept: BeautifulSoup + filtering + file

SOLUTION

```
from bs4 import BeautifulSoup

html = (
    '<h3 class="price">₹12,999</h3>'
    '<h3 class="price">₹18,500</h3>'
    '<h3 class="price">₹20,000</h3>'
)
res = BeautifulSoup(html, "html.parser")
prices = res.find_all("h3", {"class": "price"})
with open("report.txt", "w") as f:
    for p in prices:
        raw = p.text
        value = float(raw.replace("₹", "").replace(",", ""))
        if 17000 <= value <= 20000:
            f.write(raw + "\n")
```

EXPLANATION

This is a direct real-world extension of the source scraping flow.

← BACK TO INDEX

ANSWER 43 - Product title export

Source: s13.py Concept: BeautifulSoup + file writing

SOLUTION

```
from bs4 import BeautifulSoup

html = '<div class="title">Bag</div><div class="title">Shoes</div>'
res = BeautifulSoup(html, "html.parser")
titles = res.find_all("div", {"class": "title"})
with open("titles.txt", "w") as f:
    for i, item in enumerate(titles, start=1):
        f.write(f"{i}. {item.text}\n")
```

EXPLANATION

This is the same extraction-and-persistence pattern the source demonstrates.

← BACK TO INDEX

ANSWER 44 - Make the scraper resilient

Source: s13.py Concept: BeautifulSoup + data cleaning

SOLUTION

```
# Reasoned approach
PARSE -> SELECT -> CHECK matches -> PROCESS if present -> REPORT absence

Conclusion:
```

EXPLANATION

The solution follows the requirements and source concept.

INTERVIEW NOTE

Interview skill: defensive parsing and data validation

← BACK TO INDEX

ANSWER 45 - Separate raw and cleaned values

Source: s13.py Concept: String cleaning + scraping

SOLUTION

```
# Reasoned approach
KEEP raw text -> DERIVE numeric value -> USE numeric for logic

Conclusion:
```

EXPLANATION

Keeping raw text preserves presentation fidelity while the numeric value enables reliable filtering.

← BACK TO INDEX

MODULES, TIME & REGEX VALIDATION

ANSWER 46 - Import alias

Source: s10.py Concept: Modules

CORRECT ANSWER: B

The source uses hm as the alias for hi_msg.

← BACK TO INDEX

ANSWER 47 - sleep()

Source: s10.py Concept: time

CORRECT ANSWER: B

time.sleep pauses execution for the given number of seconds.

← BACK TO INDEX

ANSWER 48 - Regex character class

Source: s10.py Concept: re.search

CORRECT ANSWER: C

The range A-Z matches an uppercase alphabetic character.

← BACK TO INDEX

ANSWER 49 - Retry counter

Source: s10.py Concept: Loop logic

CORRECT ANSWER: C

The branch prints sleep mode, waits, then resets i to 1.

← BACK TO INDEX

ANSWER 50 - localtime()

Source: s10.py Concept: time.localtime

CORRECT ANSWER: B

The source accesses a[3], a[4], a[5] as local hour, minute and second.

← BACK TO INDEX

ANSWER 51 - Alias an imported function

Source: s10.py Concept: import syntax

SOLUTION

```
from mymodule import hi_msg as hm
```

EXPLANATION

This exactly matches the import alias pattern in the source.

← BACK TO INDEX

ANSWER 52 - One regex requirement

Source: s10.py Concept: Regex

SOLUTION

```
re.search(r"[0-9]", pwd)
```

EXPLANATION

The source checks the presence of a digit using [0-9].

← BACK TO INDEX

ANSWER 53 - Password rule validator

Source: s10.py Concept: regex + control flow

SOLUTION

```
import re

def validate_password(pwd):
    if len(pwd) <= 8 or len(pwd) >= 16:
        return "Invalid password!!!"
    if not re.search(r"[A-Z]", pwd):
        return "Invalid password!!!"
    if not re.search(r"[a-z]", pwd):
        return "Invalid password!!!"
    if not re.search(r"[0-9]", pwd):
        return "Invalid password!!!"
    if not re.search(r"[@#$$%^&*]", pwd):
        return "Invalid password!!!"
    return "Valid password!!!"

print(validate_password("Abc123@x"))
```

EXPLANATION

The source contains the same five independent validation checks.

← BACK TO INDEX

ANSWER 54 - Retry with sleep mode

Source: s10.py Concept: while + sleep

SOLUTION

```
from time import sleep

correct = "dev@gmail.com"
i = 1
while True:
    pwd = input("Enter password: ")
    if pwd == correct:
        print("Hey, Welcome!!!")
        break
    elif i == 3:
        print("Sleep mode...")
        sleep(2)
        i = 1
    else:
        print("Try again...")
        i += 1
```

EXPLANATION

This adapts the exact loop pattern in the source while reducing the sleep duration for a practice exercise.

← BACK TO INDEX

ANSWER 55 - Format local time

Source: s10.py Concept: time.localtime

SOLUTION

```
import time
a = time.localtime()
print(f"{a[3]:02d}:{a[4]:02d}:{a[5]:02d}")
```

EXPLANATION

The source indexes the returned structure as a[3], a[4], a[5].

← BACK TO INDEX

ANSWER 56 - Use a user-defined module cleanly

Source: s10.py Concept: modules + alias

SOLUTION

```
from mymodule import hi_msg as hm
hm("Dev")
```

EXPLANATION

Explicit imports make the dependency clear and match the source.

← BACK TO INDEX

ANSWER 57 - Account lockout guard

Source: s10.py Concept: regex + retries + sleep

SOLUTION

```
import re
from time import sleep

def valid_format(pwd):
    return (8 < len(pwd) < 16 and re.search(r"[A-Z]", pwd) and
            re.search(r"[a-z]", pwd) and re.search(r"[0-9]", pwd) and
            re.search(r"[@#$$%^&*]", pwd))

failed = 0
while True:
    pwd = input("Password: ")
    if valid_format(pwd):
        print("Accepted")
        break
    failed += 1
    print("Invalid password!!!")
    if failed == 3:
        print("Sleep mode...")
        sleep(2)
    failed = 0
```

EXPLANATION

This combines the source password validation and retry/sleep patterns into a realistic control flow.

← BACK TO INDEX

ANSWER 58 - Reusable time utility module

Source: s10.py Concept: modules + time

SOLUTION

```
# time_utils.py
import time

def get_time():
    t = time.localtime()
    return f"{t[3]:02d}:{t[4]:02d}:{t[5]:02d}"

# main.py
from time_utils import get_time as gt
print(gt())
```

EXPLANATION

This applies the user-defined module and alias patterns shown in the source.

← BACK TO INDEX

ANSWER 59 - Design the regex checks

Source: s10.py Concept: regex validation

SOLUTION


```
# Reasoned approach  
CHECK each rule -> FAIL FAST or COLLECT issues -> REPORT
```

Conclusion:

EXPLANATION

The solution follows the requirements and source concept.

INTERVIEW NOTE

Interview skill: readable validation logic

← BACK TO INDEX

ANSWER 60 - Module import choice

Source: s10.py Concept: import strategies

SOLUTION

```
# Reasoned approach  
CHOOSE minimal dependency -> IMPORT explicit symbol
```

Conclusion:

EXPLANATION

The solution follows the requirements and source concept.

← BACK TO INDEX

PANDAS & CSV DATA ANALYSIS

ANSWER 61 - Path(__file__).parent

Source: s14.py Concept: pathlib

CORRECT ANSWER: B

Path(__file__).parent locates the script directory; / joins the filename.

← BACK TO INDEX

ANSWER 62 - head() / tail()

Source: s14.py Concept: pandas inspection

CORRECT ANSWER: B

head(n) returns the first n rows.

← BACK TO INDEX

ANSWER 63 - fillna vs dropna

Source: s14.py Concept: pandas missing data

CORRECT ANSWER: B

fillna(0) replaces missing values with zero.

← BACK TO INDEX

ANSWER 64 - loc vs iloc

Source: s14.py Concept: pandas indexing

CORRECT ANSWER: B

The source notes this distinction explicitly.

← BACK TO INDEX

ANSWER 65 - apply(lambda)

Source: s14.py Concept: pandas derived column

CORRECT ANSWER: B

apply runs the lambda for each value in Total and builds a new Result column.

← BACK TO INDEX

ANSWER 66 - Create Total column

Source: s14.py Concept: pandas arithmetic

SOLUTION

```
data["Total"] = data["Physics"] + data["Maths"] + data["Chemistry"]
```

EXPLANATION

The source uses this exact vectorized column calculation.

← BACK TO INDEX

ANSWER 67 - Difference between loc and iloc

Source: s14.py Concept: pandas indexing

SOLUTION

See the worked solution.

EXPLANATION

This is explicitly explained in the source comments.

← BACK TO INDEX

ANSWER 68 - Clean and classify marks

Source: s14.py Concept: pandas + missing data + apply

SOLUTION

```
data = data.fillna(0)
data["Total"] = data["Physics"] + data["Maths"] + data["Chemistry"]
data["Result"] = data["Total"].apply(lambda n: "pass" if n > 200 else "fail")
```

EXPLANATION

This mirrors the source transformation pipeline.

← BACK TO INDEX

ANSWER 69 - Filter failing Maths rows

Source: s14.py Concept: pandas loc

SOLUTION

```
filtered = data.loc[(data["Maths"] > 30) & (data["Result"] == "fail")]
```

EXPLANATION

The source gives this exact boolean filtering pattern with loc.

← BACK TO INDEX

ANSWER 70 - Remove duplicate records

Source: s14.py Concept: pandas duplicates

SOLUTION

```
print(data.duplicated())
data = data.drop_duplicates()
```

EXPLANATION

The source explicitly demonstrates both operations.

← BACK TO INDEX

ANSWER 71 - Export cleaned results

Source: s14.py Concept: pandas to_csv

SOLUTION

```
data.to_csv("new.csv", index=False)
```

EXPLANATION

The source comments describe creating a new CSV from the modified data.

← BACK TO INDEX

ANSWER 72 - Student performance report

Source: s14.py + marks.csv Concept: pandas + CSV

SOLUTION

```
from pathlib import Path
import pandas as pd

path = Path("marks.csv")
data = pd.read_csv(path)
print(data.head(2))
data = data.fillna(0)
data["Total"] = data["Physics"] + data["Maths"] + data["Chemistry"]
data["Result"] = data["Total"].apply(lambda n: "pass" if n > 200 else "fail")
data.to_csv("cleaned_marks.csv", index=False)
```

EXPLANATION

This combines the exact data-loading, cleaning, derivation and export operations shown in the source.

← BACK TO INDEX

ANSWER 73 - Academic failure filter

Source: s14.py Concept: pandas loc + derived column

SOLUTION

```
filtered = data.loc[(data["Maths"] > 30) & (data["Result"] == "fail")]
```

EXPLANATION

This is the exact filtering pattern shown in the source comments.

← BACK TO INDEX

ANSWER 74 - Fill or drop missing data?

Source: s14.py + marks.csv Concept: fillna / dropna

SOLUTION

```
# Reasoned approach
UNDERSTAND missingness -> CHOOSE fill or drop -> VALIDATE impact

Conclusion:
```

EXPLANATION

The solution follows the requirements and source concept.

INTERVIEW NOTE

Interview skill: data-quality reasoning

← BACK TO INDEX

ANSWER 75 - Vectorized column calculation vs row loop

Source: s14.py Concept: pandas

SOLUTION

```
# Reasoned approach
USE column operations for vector math -> APPLY only for custom element-wise rule

Conclusion:
```

EXPLANATION

The solution follows the requirements and source concept.

INTERVIEW NOTE

Pattern: vectorize simple arithmetic; apply for custom transformations

← BACK TO INDEX

FINAL REVIEW GUIDE

How to use this workbook for interview and coding practice

INTERVIEW

Before an interview, revisit the concept notes and attempt the problem questions without code first. Be ready to explain why you chose a particular operation, not only what the code does.

LEETCODE FOUNDATION

The source-backed questions emphasize transferable patterns already present in the materials: transformation pipelines, filtering, nested data access, parsing, searching, validation, and dataframe filtering. Use the thinking questions to practice identifying the operation before coding.

DEBUGGING CHECKLIST

1. Confirm input type. 2. Inspect intermediate values. 3. Separate parsing from processing. 4. Test boundary cases. 5. Compare actual output with the stated expected output. 6. Only then optimize.